FastAPI Backend

`backend/main.py` is a **read-only serving layer** for the NIDS. It sits *off* the hot path: it never inspects traffic or scores packets. It only exposes what the rest of the pipeline has already stored, so a dashboard/SOC UI has something to call.



Three independent data sources feed it:

Source	Used for	Client in code
OpenSearch	Historical alerts, filtering, aggregations	`os_client` (`main.py:56`)
MinIO / S3	Listing Parquet files in the data lake	`s3_client` (`main.py:64`)
Kafka `nids-alerts`	Live push feed over WebSocket	`AIOKafkaConsumer` (`main.py:236`)

How it's wired

- One `FastAPI` app: `main.py:54`, started with `uvicorn backend.main:app` (see `docker-compose.yml`, service `backend`).
- Config is not duplicated: it imports from `../services/storage_common.py` after adding that folder to `sys.path` (`main.py:35`). All values (Kafka brokers, OpenSearch host, bucket, index prefix) are env-var overridable.
- OpenSearch and boto3 clients are created **once at import time**, not per request. Both are synchronous/blocking libraries.
- Sync endpoint functions (`def`, not `async def`) are automatically run in FastAPI's threadpool, so those blocking calls do not stall the event loop. Only the WebSocket code is `async`.

Endpoints

Endpoint	What it does	Backing query
`GET /health`	Pings OpenSearch cluster + MinIO bucket, returns `{opensearch: bool, minio: bool}`	`main.py:76`

Endpoint	What it does	Backing query
`GET /alerts`	Recent alerts, newest first. Optional `severity_tier`, `src_ip`, `dst_ip`, `since_minutes`, `limit`; filters are AND-combined	`bool.must` range on `@timestamp` + `term` filters over `nids-alerts-*` (`main.py:94`)
`GET /alerts/{doc_id}`	One alert by its OpenSearch `_id`; 404 if missing	`ids` query (`main.py:132`)
`GET /stats/severity`	Alert counts per severity tier in a time window	`terms` aggregation, `size: 0` (`main.py:151`)
`GET /datalake/objects`	Lists Parquet files for a day, default today UTC	`list_objects_v2` under `engineered-features/dt=YYYY-MM-DD/` (`main.py:171`)
`WS /ws/alerts`	Push-only live alert feed	Kafka bridge (`main.py:222`)

Times are handled as relative OpenSearch ranges (`now-{N}m`), so the caller never sends absolute dates. Indices are daily (`nids-alerts-YYYY.MM.DD`, created by `alert_indexer.py`), and the wildcard `nids-alerts-*` searches across all of them.

The WebSocket part (the only non-trivial bit)

1. `ConnectionManager` (`main.py:196`) keeps a plain list of connected sockets and broadcasts to all of them.
2. `kafka_to_websocket_bridge` (`main.py:234`) is a background consumer on `nids-alerts` (`group_id="backend-ws-bridge"`, `auto_offset_reset="latest"`) that forwards every message to every socket.
3. It is launched from the `@app.on_event("startup")` hook as a fire-and-forget `asyncio.create_task` (`main.py:252`).
4. The WS handler itself is push-only: it waits on `receive_text()` purely so a client disconnect raises `WebSocketDisconnect` and the socket gets cleaned up.
5. This path deliberately bypasses OpenSearch, so a dashboard sees alerts immediately instead of after the indexer's ~5s batch.

How to run / test

```
docker compose up -d backend        # then open http://localhost:8000/docs
# or locally:
uvicorn backend.main:app --port 8000
```

Interactive Swagger docs are at `/docs`; the WebSocket can be tested with `wscat` or any WS client.

Testing with Postman

Prerequisites

1. Start the stack and make sure data exists:

```
docker compose up -d
```

2. Open Postman, click **Import**, and paste `http://localhost:8000/openapi.json`. Postman generates a request collection for every endpoint automatically.

Base URL for all requests: `http://localhost:8000` (no auth headers needed).

1. `GET /health`

```
GET http://localhost:8000/health
```

No params. Expected:

```
{ "opensearch": true, "minio": true }
```

If either value is `false`, the backend cannot reach that service - fix that before testing the rest.

2. `GET /alerts`

```
GET http://localhost:8000/alerts
```

All params are optional. Add them in Postman's **Params** tab (they become `?key=value`):

Key	Example	Notes
`severity_tier`	`HIGH`	CRITICAL / HIGH / MEDIUM / LOW; auto-uppercased
`src_ip`	`192.168.1.10`	exact term match
`dst_ip`	`10.0.0.5`	exact term match
`since_minutes`	`1440`	1 to 43200 (30 days), default 60
`limit`	`10`	1 to 1000, default 50

Example:

```
GET http://localhost:8000/alerts?severity_tier=HIGH&since_minutes=1440&limit=5
```

Response shape:

```
{
  "total": 5,
  "alerts": [
    {
      "id": "A1b2C3...",
      "severity_tier": "HIGH",
      "@timestamp": "2026-10-28T12:34:56Z",
      "src_ip": "192.168.1.10",
      "dst_ip": "10.0.0.5",
      "score": 0.87,
      "...": "rest of the fused alert document"
    }
  ]
}
```

`total` is the number of hits returned (bounded by `limit`), not the full match count.

3. `GET /alerts/{doc_id}`

Copy an `id` from step 2 and put it in the path:

```
GET http://localhost:8000/alerts/A1b2C3...
```

- Found -> single alert object.
- Not found -> `404 {"detail": "Alert not found"}`.

4. `GET /stats/severity`

```
GET http://localhost:8000/stats/severity?since_minutes=1440
```

Response is a plain map of tier to count:

```
{ "CRITICAL": 2, "HIGH": 14, "MEDIUM": 53, "LOW": 210 }
```

5. `GET /datalake/objects`

```
GET http://localhost:8000/datalake/objects?date=2026-10-28&limit=100
```

`date` must be `YYYY-MM-DD`; omit it to use today (UTC). Response:

```
{
  "bucket": "nids-datalake",
  "prefix": "engineered-features/dt=2026-10-28/",
  "count": 2,
  "objects": [
    {
      "key": "engineered-features/dt=2026-10-28/hour=12/part-abc.parquet",
      "size_bytes": 48213,
      "last_modified": "2026-10-28T12:05:11+00:00"
    }
  ]
}
```

An empty `objects` list means no data was written for that day yet.

6. WebSocket `WS /ws/alerts`

In Postman: **New -> WebSocket** (not HTTP), enter:

```
ws://localhost:8000/ws/alerts
```

Click **Connect**. Alerts appear automatically as the pipeline emits them - you do not send a request. Anything you type and send is ignored by the server; it is push-only. If nothing arrives, generate traffic through the producers or publish to `nids-alerts`.

Watching the pipeline while testing

- Kafka messages: `docker compose exec kafka ...` or the Kafka UI at `http://localhost:8080`
- Indexed alerts: OpenSearch Dashboards at `http://localhost:5601`
- Parquet files: MinIO console at `http://localhost:9501`

Troubleshooting

Symptom	Cause
`502 OpenSearch query failed`	Backend can reach OpenSearch but the `nids-alerts-*` index does not exist yet, or `alert-indexer` is not running. Check `/health` and the `alert-indexer` container.
`/alerts` returns `total: 0`	No alerts inside `since_minutes`. Raise it to `1440` or `43200`.
`/datalake/objects` returns `count: 0`	Wrong `date`, or `datalake-writer` has not flushed a batch yet (default flush is 2000 rows / 60s).
`502 MinIO list failed`	`minio-init` has not created the bucket, or MinIO is unhealthy.

Symptom	Cause
Postman shows connection refused	Backend is not up - check `docker compose ps` and `http://localhost:8000/health`.

Real caveats in this code

- `@app.on_event("startup")` is **deprecated** in FastAPI 0.111 - the `lifespan` context manager is the current pattern. It works, but it is the one thing worth changing first.
- The bridge task is **unsupervised**: if Kafka is down at boot, `consumer.start()` throws inside the task and the live feed silently stays dead until restart. No retry, and no shutdown hook to `consumer.stop()`.
- `/health` always returns HTTP 200 even when both backends are down; a load balancer would consider it healthy. It also blocks a threadpool worker on two network calls.
- No auth, no CORS, no rate limiting - anything that can reach port 8000 can read all alerts.
- If no daily index exists yet, `nids-alerts-*` matches nothing and OpenSearch errors, so the endpoint returns 502 (not an empty list).
- There is a duplicate fork at `kafka-process/backend/main.py`; edit only one or they will drift.